# Object Oriented Programming Language

# OPERATOR OVERLOADING

- For OOP practicals, the following progression is recommended:
- void Add() – simplest, demonstrates a member function with no arguments.
- Complex Add(Complex c) – demonstrates passing an object as an argument (most practical design).
- Complex Add(Complex c1, Complex c2) – demonstrates passing multiple objects.
- friend Complex Add(Complex c1, Complex c2) – demonstrates friend functions.

```
#include <iostream>
using namespace std;
class Complex
{
   int real, imag;
public:
   void accept()
   {
      cin >> real >> imag;
   }
   Complex Add();
   void display()
   {
      cout << real << " + " << imag <<
"i";
   }
};
Complex c2;    // Global object
```

```
Complex Complex::Add()
{
   Complex temp;
   temp.real = real + c2.real;
   temp.imag = imag + c2.imag;
   return temp;
}
   int main()
   {
      Complex c1, c3;
      cout << "Enter first complex
   number: ";
      c1.accept();
      cout << "Enter second complex
   number: ";
      c2.accept();
      c3 = c1.Add();
      cout << "Addition = ";
      c3.display();
      return 0;
   }
```

```
#include <iostream>
using namespace std;
class Complex
{
private:
   int real, imag;
public:
   void accept()
   {
      cout << "Enter Real and Imaginary Part: ";
      cin >> real >> imag;
   }
   Complex Add(Complex c)
   {
      Complex temp;
      temp.real = real + c.real;
      temp.imag = imag + c.imag;
      return temp;
   }
   void display()
   {  cout << real << " + " << imag << "i" <<
endl;    }
};
```

```
int main()
{
   Complex c1, c2, c3;

   cout << "Enter First Complex
Number:\n";
   c1.accept();

   cout << "Enter Second Complex
Number:\n";
   c2.accept();

   c3 = c1.Add(c2);

   cout << "Addition = ";
   c3.display();

   return 0;
}
```

```
#include <iostream>
using namespace std;

class Complex
{
private:
   int real, imag;

public:
   void accept()
   {
      cout << "Enter Real and Imaginary Part: ";
      cin >> real >> imag;
   }

   Complex Add(Complex c1, Complex c2)
   {
      Complex temp;
      temp.real = c1.real + c2.real;
      temp.imag = c1.imag + c2.imag;
      return temp;
   }
```

```
   void display()
   {
      cout << real << " + " << imag << "i"
<< endl;
   }
};

int main()
{
   Complex c1, c2, c3;

   cout << "Enter First Complex
Number:\n";
   c1.accept();

   cout << "Enter Second Complex
Number:\n";
   c2.accept();
   c3 = c3.Add(c1, c2);
   cout << "Addition = ";
   c3.display();

   return 0;
}
```

# Friend Function Example

```
#include <iostream>
using namespace std;
class Student
{
private:
   int marks;
public:
   Student()
   {
      marks = 90;
   }
   friend void display(Student);
};
```

```
void display(Student s)
{
   cout << "Marks = " << s.marks;
}

int main()
{
   Student s;
   display(s);

   return 0;
}
```

```
#include <iostream>
using namespace std;

class Complex
{
private:
   int real, imag;

public:
   void accept()
   {
      cout << "Enter Real and Imaginary Part: ";
      cin >> real >> imag;
   }

   void display()
   {
      cout << real << " + " << imag << "i" <<
endl;
   }

   friend Complex Add(Complex, Complex);
};
```

```
Complex Add(Complex c1, Complex c2)
{
   Complex temp;

   temp.real = c1.real + c2.real;
   temp.imag = c1.imag + c2.imag;

   return temp;
}

int main()
{
   Complex c1, c2, c3;

   cout << "Enter First Complex Number:\n";
   c1.accept();

   cout << "Enter Second Complex Number:\n";
   c2.accept();

   c3 = Add(c1, c2);

   cout << "Addition = ";
   c3.display();

   return 0;
}
```

# Friend Function

# Topics To Be Covered

- Introduction
- Need of operator overloading
- Overloading the assignment
- Binary and unary operators
- Overloading using friends
- Rules for operator overloading
- Type conversions

# Introduction

- Operator Overloading provides a flexible options for the **creation of new definitions** for most of the C++ operators.
- Can you List down few operators that we can overload in C++ ?
- We can Overload almost all the operators in C++ except the following operators :

1) Class member access operatos (., .*)
2) Scope Resolution Operator (::)
3) SizeOf Operator (sizeof)
4) Conditional Operators (?:)

# Introduction to Polymorphism

- Polymorphism means **more than one form**. That is, the same entity (function or operator) behaves differently in different scenarios.

**Compile time polymorphism / Static Binding / Early Binding**

- binding is occuring at compile time
- Inheritance is not involved

**Run time polymorphism / Dynamic Binding / late Binding**

- at run time we came to know which method is going to invoke
- Inheritance is not involved

# Introduction to Polymorphism

## Function overloading:

- Two or more functions having the **same name** but **different parameters [Refer Unit-2]**

## Operator overloading:

- It is a **compile-time polymorphism** in which the operator is overloaded to provide the **special meaning to the user-defined data type**
- In other words, giving the C++ operators such as +, *, == additional meanings when they are applied with user-defined data types.

# Introduction to Polymorphism

```
int a=5, b=10,c;
c = a + b;
```

Operator + performs **addition** of **integer operands** a, b

```
Time t1,t2,t3;
t3 = t1 + t2;
```

Operator + performs **addition** of **objects** of type **Time**

```
string str1="Hello"
string str2="Good Day";
string str3;
str3 = str1 + str2;
```

Operator + **concatenates** two strings str1,str2

# Introduction to Polymorphism

**Any symbol can be used as function name**

- If it is a valid operator in C language
- If it is preceded by operator keyword

**We can overload all the C++ operators except the following:**

- Scope resolution operator (::)
- Size operator(sizeof)
- member selection (.)
- member selection with pointer-to-member(.*)
- Ternary/conditional operator(?:)

# Introduction to Polymorphism

- C++ operators that can be overloaded

| Operators that can be overloaded | | | | | | | |
|---|---|---|---|---|---|---|---|
| + | - | * | / | % | ^ | & | \| |
| ~ | ! | = | < | > | += | -= | *= |
| /= | %= | ^= | &= | \|= | << | >> | >>= |
| <<= | == | != | <= | >= | && | \|\| | ++ |
| -- | ->* | , | -> | [] | () | new | delete |
| new[] | delete[] | | | | | | |

- C++ Operators that cannot be overloaded

| Operators that cannot be overloaded | | | | |
|---|---|---|---|---|
| . | .* | :: | ?: | sizeof |

# Defining Operator Overloading

- **The general form of an operator function is:**




**NOTE:**

Operator functions must be either member functions(non-static) or friend functions.

- **Friend function** will have only **one** argument for **unary** operators and **two** for **binary** operators.
- **Member function** has **no** argument for **unary** operators and only **one** for **binary** operators.

- Operators cannot be overloaded if they are essential to the compiler for parsing the program, accessing object members, determining types or sizes, or defining the core syntax of the C++ language. Allowing them to be overloaded would make the language ambiguous and unreliable.

# Why these operators cannot be overloaded

**1. They are compile-time operations**

Some operators are evaluated by the compiler before the program runs.

Example: `int x; cout << sizeof(x);`

The compiler replaces `sizeof(x)` with `4` (or `8`, depending on the system). Since there is no runtime operation, overloading is impossible.

**2. They define the language syntax**

Example: `obj.display();` The `.` operator tells the compiler to access a member of `obj`.

If programmers could overload `.`, then `obj.display();` could perform anything making programs confusing and difficult for the compiler to understand.

**3. They affect parsing of the program**

Example:

`A::B`

The compiler uses `::` while parsing the program to locate `B` inside `A`.

If `::` were overloadable, the compiler would not know whether it represents namespace/class access or a user-defined operation.

# Why these operators cannot be overloaded

**4. They have fixed evaluation rules**
Example:

```
result = condition ? x : y;
```

The ternary operator evaluates only one of `x` or `y`.

If overloaded, this behavior could change, breaking the language's guarantees.

**Example**
Suppose C++ allowed overloading of `.`:

```
Student s;
s.marks;
```

A programmer might redefine `.` to:

- print the marks,
- delete the object,
- open a file,
- or perform any arbitrary action.

This would make code unpredictable and difficult to read.

# Restrictions on Operator Overloading

- Arity (number of operands) cannot be changed
- **No new operators** can be created (Use only existing operators)
- You cannot change how two integers are added

# Operator Overloading Syntax

return type classname :: **operator** op (arglist)

{

// Function Body

}

- As shown above the operator to be overloaded (op) is preceded by the keyword **operator .**

# Process of overloading

1) Create a **class that defines** the data type that is to be overloaded.
2) Declare the operator function **operator op()** in the public part of the class. It may be either a member function or a non-member function.
3) Define the operator function to implement the required operations.

# Overloading Unary Operator

- Unary operators have only one operand.

```
#include <iostream.h>

Class space
{
    int x,y,z;
    public :
        void getdata(int a, int b, int c);
        void display();
        void operator – ();          // overload unary minus
}
```

```
Void space :: getdata(int a, int b, int c)
{
    x=a; y=b; z=c;
}
Void space :: display()
{
    cout << x << “” << y << “ “ << z << “\n”;
}
void space :: operator - ()
{
    x = -x ;          y = -y;   z = -z;
}
```

```
Int main()
{
    space s;
    s.getdata(10,-20,30);
    cout << “s : “ ;
    s.display();
    -S;
    cout << “s : “;
    s.display();
    return 0;
}
```

# Overloading Binary Operators

```
#include <iostream>
class complex
{
   float x, y;
   public :
            complex () {    }
            complex (float real, float imag)
              {x = real;              y = imag;}
            complex operator + (complex);
            void display();
};
```

```
Complex complex :: operator + (complex c)
{
    complex temp;
    temp.x = x + c.x;
    temp.y = y + c.y;
    return temp;
}
Void complex :: display()
{
        cout << x << “+j” << y << “\n” ;
}
```

```
Int main()
{
    complex c1(2.5,3.5),c2(1.6,3.5),c3;
    c3 = c1 + c2;
    cout << “c1 = ” << c1.display();
    cout << “c2 = ” << c2.display();
    cout << “c3 = ” << c3.display();
}
```

Output :

C1 = 2.5 + j3.5

C2 = 1.6 + j2.7

C3 = 4.1 +j6.2

The operator overloading function that was written is as below :-

```
Complex operator + (complex c)
{
    complex temp;
    temp.x= x+c.x;
    temp.y = y+c.y;
    temp.z = z+c.z;
    return temp;
}
```

The above function has only one argument. Why ????

ANS: The member function can be invoked by an object of the same class. In the case C3 = C1 + C2 is equivalent to

c3 = c1.operator+(c2).

# Operator as a Friend

- Member functions
  - Use **this** keyword to implicitly get argument
  - Gets left operand for binary operators (like +)
- Non member functions (can be a friend function)
  - Need parameters for both operands
  - Can have object of different class than operator
  - Must be a **friend** to access **private** or **protected** data

- An operator implemented as a friend function must explicitly have arguments of the object.

- Overloading << and >> operators
  - Left operand of type **ostream &**
    - Such as **cout** object in **cout << classObject**
  - Similarly, overloaded >> needs **istream &**

Thus, both overloading functions (for >> and << ) must be non-member functions.

# Overloading Binary Operators Using Friend

```
#include <iostream.h>
Const size = 3;
Class vector
{   int v[size];
    public :
            vector();
            vector(int *x);
            friend vector operator *(int a, vector b);
            friend vector operator *(vector b,int a);
            friend istream &operator >> (istream &, vector &);
            friend istream &operator << (ostream &, vector &);
};
```

```
Vector :: vector ()
{   for (int i =0 ; i<size ; i++)
            v[ i ] = 0;                          }
Vector :: vector (int *x)
{   for (int I =0; I < size ; i++)
            v[ i ] = x[ i ];                     }
Vector operator *(int a,vector b)
{   vector c;
    for (int i = 0; i<size; i++)
            c.v [ i ] = a *  b.v[ i ];
    return c;                                    }
Vector operator *(vector b, int a)
{   vector c;
    for (int i = 0; i<size; i++)
            c.v [ i ] = b.v[ i ] * a;
    return c;                                    }
```

```
Istream & operator >> (istream &din, vector &b)
{
    for (int i = 0; i<size; i++)
        din >> b.v[i];
    return (din);
}
Ostream & operator << (ostream &dout, vector &b)
{
    dout <“(“ << b.v[0];
    for (int I = 0;i<size; i++)
        dout << “,” << b.v[i];
    dout << “)”;
    return (dout);
}
```

```
int x[size] = {2,4,5};
int main()
{
    vector m;
    vector n=x;
    cout << “Enter elements of vector m” << “\n”;
    cin >> m;
    cout << “\n”;
    cout << “m=“ << m << “\n”;
    vector p,q;
    p = 2*m;       q=n*2;
    cout << “\n”;
    cout << “p=“ << p <<“\n”;
    cout << “q = “ <<q <<“\n”;
}
```

Output :

Enter elements of vector m

5 10 15

m = (5,10,15)

p = (10, 20, 30)

q = (4, 8, 12)

Prac3 :-

Design a Class Complex with data members for real and imaginary part. Provide default and parameterized constructors. Write a program to perform arithmetic operations of two complex numbers using operator overloading (using either member functions or friend functions. **Strictly use Operator Overloading.**

# Manipulation of Strings Using Operators

- Some drawbacks of string manipulations in C are :-
  - There are no operators for manipulating the strings.
  - One of the main drawback is that whenever a string is to be copied, the programmer must first determine its length and allocate the required amount of memory.

  - The above mentioned drawbacks exists in C++ as well. But here, c++ allows us to create our own definitions of operators that can be used to manipulate strings.

**NOTE:** Since strings vary greatly in size, we use ***new*** to allocate memory for each string and a pointer variable to point to the string array.

```
#include <iostream.h>
#include <string.h>

Class string
{
    char *p;
    int len;
    public :
```

```
        string()        {len=0; p=0;        }
        string(char *s);
        string (string &s);
        ~ string()    { delete p;   }
        friend string operator + (const string &s, const
    string &t);
        friend int operator <= (string &s, string &t);
        friend void show(string s);
};
```

```
String :: string(char *s)
{       len = strlen(s);
        p = new char[len+1];
        strcpy(p,s);                    }
String :: string (string &s)
{       len = s.len;
        p = new char[len+1];
        strcpy(p,s.p);                  }
```

```
String operator +(string &s, string &t)
{       string temp;
        temp.len = s.len + t.len;
        temp.p = new char[temp+1];
        strcpy(temp.p, s.p);
        strcat (temp.p, t.p);
        return (temp);
}
```

```
Int operator <= (string &s, string &t)
{
    int m = strlen(s.p);
    int n = strlen(t.p);

    if (m<=n)   return (1);
    else                    return (0);
}
Void show(string s)
{
    cout << s.p;
}
```

```
int main()
{   string s1 = “new”;      string s2 = “york”;          string s3 = “delhi”;
    string t1, t2, t3;
    t1 = s1;          t2 = s2;  t3 = s1 + s2;
    cout << “\n t1 = “; show (t1);
    cout << “\n t2 = “; show (t2);     cout << “\n”;
    cout  << “\n t3 =“; show (t3);     cout << ‘\n\n”;
    if (t1 < = t3)
    {       show(t1);          cout << “smaller than”;     show(t3);
            cout << “\n”;
    }
    else
    {       show(t3);          cout << “smaller than”;     show(t1);
            cout << “\n”;
    }
    return 0;
}
```

OUTPUT :

t1 = new

t2 = york

t3 = new delhi

new smaller than new delhi

Practical 4:

- Write a C++ program to perform String operations
  - == Equality
  - = String Copy
  - + Concatenation
  - << To display a string
  - >> To reverse a string
  - Function to determine whether a string is a palindrome.
  - To find occurrence of a sub-string.
- Strictly Use Operator Overloading.

```
friend String operator  + (String&,String&);
{     String temp;
      temp.len = s1.len+s2.len;
      temp.str=new char[temp.len+1];
      strcpy(temp.str,s1.str);
      strcat(temp.str,s2.str);
      return temp;
}
```

```
friend int operator==(String&,String&);
        //equality
    {
        int cmp_result;
        cmp_result=strcmp(s1.str,s2.str);
        return cmp_result;
    }
```

```
int operator!(String& s1)
{
    char*ptr1,*ptr2;
    ptr1=s1.str;
    ptr2=s1.str+s1.len-1;
    while(ptr1<=ptr2)
    {
        if(*ptr1!=*ptr2)
        return 0;
        else
        {       ptr1++;         ptr2--;
        }
    }
    return 1;
}
```

```
friend void operator*(String&,String&);//      strcpy
friend void operator<<(String&,String&);//   strrev
friend istream& operator>>(istream&,String&);//Input
friend ostream& operator<<(ostream&,String&);//Output
```

**Find and Replace**

```
void operator/(String& s1,char ch1)
{
    char*c=s1.str;              char ch2;
    cout<<"\nEnter Character to replace:";
    cin>>ch2;
    while(*c!='\0')
    {
        if(*c==ch1)
        {           *c=ch2;
        }           c++;
    }
}
```

friend void operator/(String&,char);  //Find char and replace

→ Try **strstr** function .

# TYPE CONVERSION

3 scenarios might arise :-

- Conversion from basic type to class type.
- Conversion from class type to basic type.
- Conversion from one class type to another class type.

# Basic to Class Type

```
String :: string (char *a)
{
    len = strlen(a);
    p = new char[len + 1];
    strcpy(p,a);
}
```

```
String s1,s2;
Char *name = “hello”;   char *name2 = “world”;
S1 = string(name);          OR        S2 = name2;
```

Note : in the previous conversions, we have used constructors.

The constructors used for conversions take single argument whose type is to be converted.

# Class To Basic Type

- Constructors did a fine job in converting from basic type to class type, but they cannot perform conversions from class to basic.
- C++ allows us to define an **overloaded casting operator** that could be used to convert a class type into basic type.
- Such overloaded casting operator function are also referred to as **conversion function.**

Operator typename()

{

…. (function statement)

}

- Note : The casting operator function should satisfy the following conditions ;
  - It must be a class member.
  - It must not specify a return type.
  - It must not have any arguments.

```
String :: operator char* ()
{
   return (p);
}
```

```
Vector :: operator double()
{
    double sum = 0;
    for (int i=0; i< size ; i++)
    {
        sum = sum + v[i] * v[i];
    }
}

Double length = double(v1);
        OR
Double length = v1;
```

- When the compiler encounters  a statement that requires a conversion of a class type to a basic type, it quietly calls a casting operator to do the job.
- Since it is a member function, it is invoked by the object and therefore, the values used for the conversion inside the function belong to the object that invoked the function.

# One class to Another Class

- It can be carried out either by a constructor or by writing an overloaded operator function.
- Eg : obj x = obj y means object of classY has to be converted to an object of class x and then assigned to obj x.
- In above statement classX is called the Destination class and classY is called the Source Class

  - Write a conversion overloaded operator function in Source Class (class Y)

    OR

  - Write a constructor in Destination Class (Class X)

```
#include<iostream.h>

Class invent1
{
    int code;       int items;          float price;
    public :
        invent1(int a, int b, intc)
        {       code =a;            Items=b;            price=c;
        }
        void putdata()
        {
                cout << “code: “ << code << “\n”;
                cout << “items :” <<items <<“\n”;
                cout << “value : “<< price << “\n”;
        }
```

```
        int getcode()      { return code;    }
        int getitems()     { return items;   }
        float getprice()   { return price;   }
        operator float()   { return (items *price);   }

/*  operator invent2()
    {
        invent2 temp;
        temp.code = code;
        temp.value = price * items;
        return temp;
    }
*/
    };     // END OF SOURCE CLASS
```

```
Class invent2                    // DESTINATION CLASS
{
    int code;        float value;
    public:
        invent2()
        {                        code = 0; value=0;
        }
        invent2 (int x, float y)
        {                        code =x; value = y;
        }
        Void putdata()
        {           cout << “code : “ << code << “\n”;
                    cout << “value : “ << value << “\n\n”;
        }
```

```
        invent2 (invent1 p)        // CONVERSION CONSTRUCTOR
        {          code = p.getcode();
                   value = p.getitems() * p.getprice();
        }
};

int main()
{   invent1 s1(100,5,140.0);
    invent2 d1;                    float total_value;

    total_value = s1;        // INVENT1 TO FLOAT
    d1=s1;                   // INVENT1 TO INVENT2

    cout << “ProductDetails – invent1 type” << “\n”;
    s1.putdata();
```

```
    cout << '\n stock value " << "\n";
    cout <<  "value = " << total_value << "\n\n";
    cout << 'ProductDetails – invent2 type " << "\n";
    d1.putdata();
    return 0;
}
```

O/P :

Product details – invent1 type

Code : 100

Items : 5

Value : 140

Stock value

Value = 700

Product details- invent2 type

Code : 100

Value : 700

# Rules for Operator Overloading

- Only existing operators can be overloaded.
- The overloaded operators must have atleast one operand that is of user defined type.
- We cannot change the basic meaning of an operator.  i.e. we cannot redefine plus(+) to subtract one value from other.
- Overloaded operators follow the syntax rules of the original operators. They cannot be overridden.
- We cannot use friend functions to overload certain operators.

- A friend function cannot be used with the following operators :-
  - “=“ : assignment operator
  - “()” : function call operator
  - “[]” : subscripting operator
  - “->” : class member access operator

THANK YOU